Numerisk Analys av uppgift 1

Kurs: HI1029 Algoritmer och datastrukturer

Namn: Ibrahim Awad

Inledning

I denna uppgift analyserar jag en kod med fyra nästlade loopar. Målet är att ta reda på hur många gånger satsen r++ körs beroende på värdet n. Denna funktion kallar jag $T(n)$. Analysen görs med numerisk metod genom att studera resultat för olika värden på n.

Metod

Jag bestämmer tidsfunktionen $T(n)$ med hjälp av numerisk analys i tre steg. Jag börjar med att köra programmet för flera olika värden på n. För varje värde räknar jag hur många gånger satsen r++ utförs.

Exempel på erhålla värden visas i tabellen nedan:

T(n)	Värdet
1	1 gånger
2	7 gånger
10	1705 gånger
20	23 485 gånger

I det första försöket anpassar jag polynom av stigande grad till de numeriska värdena på $T(n)$. Jag börjar med polynom av låg grad och ökar graden steg för steg. För varje polynom jämför jag hur bra det passar de värden jag har räknat fram.

I det andra försöket använder jag den lägsta grad som visat sig fungera väl och anpassar ett polynom av just denna grad till samtliga datapunkter. Detta polynom betraktar jag som en tidsfunktionen $T(n)$.

I det tredje försöket skriver jag om polynomets koefficienter till bråkform genom att tolka decimalvärdena och bortse från små avrundningsfel. Slutligen verifierar jag resultatet genom att beräkna $T(1000)$både genom direkt körning av programmet och genom att använda polynomet, och jämför dessa värden.

Resultat

När jag kör programmet för olika värden på $n$får jag en växande följd av värden på $T(n)$. Tabellen visar tydligt att antalet exekveringar ökar snabbt när n blir större.

När jag använder ett polynom av grad 3 ser jag att polynomet följer trenden men inte passar värdena tillräckligt bra, särskilt för större värden på n. Detta visar att grad 3 inte räcker.

När jag i stället använder ett polynom av grad 4 följer polynomet värdena mycket bättre. Det finns inga tydliga skillnader mellan polynomet och de beräknade värdena. Därför drar jag slutsatsen att grad 4 är den lägsta grad som behövs.

När jag skriver om polynomets koefficienter till bråkform blir bråken ganska stora. Detta beror på avrundningsfel i den numeriska anpassningen och är förväntat.

För att kontrollera resultatet beräknar jag $T(1000)$ genom att köra programmet direkt. Då får jag

$$T(1000) = 125\,417\,041\,750$$

När jag använder polynomet för att beräkna $T(1000)$ får jag ett värde som skiljer sig från det exakta. Skillnaden beror på avrundningsfel och på att polynomet används utanför det område där data samlades in. Detta påverkar dock inte slutsatsen.

Slutsats

Den numeriska analysen visar att tidsfunktionen för den givna koden växer mycket snabbt när $n$blir större. Genom att anpassa polynom av olika grad ser vi att ett polynom av grad 4 är det lägsta som kan beskriva tillväxten på ett bra sätt.

Detta innebär att antalet exekveringar ökar proportionellt mot n^4för stora värden på n. Därför är kodens tidskomplexitet av ordningen **O(n4)**